前言

首先来看一下C++的介绍，百度百科C++ -- >点我跳转

> C++是在C的基础之上，容纳进去了面向对象编程思想，并增加了许多有用的库，以及编程范式等。熟悉C语言之后，对C++学习有一定的帮助，本章节主要目标：
>
> 1. 补充C语言语法的不足，以及C++是如何对C语言设计不合理的地方进行优化的，比如：作用域方面、IO方面、函数方面、指针方面、宏方面等。
> 2. 为后续类和对象学习打基础。

C语言有关键字，那么C++也有关键字，首先我们来认识一下这些**关键字**

一、C++关键字(C++98)

C++总计**63**个关键字，C语言**32**个关键字

asm	do	if	return	try	continue
auto	double	inline	short	typedef	for
bool	dynamic_cast	int	signed	typeid	public
break	else	long	sizeof	typename	throw
case	enum	mutable	static	union	wchar_t
catch	explicit	namespace	static_cast	unsigned	default
char	export	new	struct	using	friend
class	extern	operator	switch	virtual	register
const	false	private	template	void	true
const_cast	float	protected	this	volatile	while
delete	goto	reinterpret_cast			

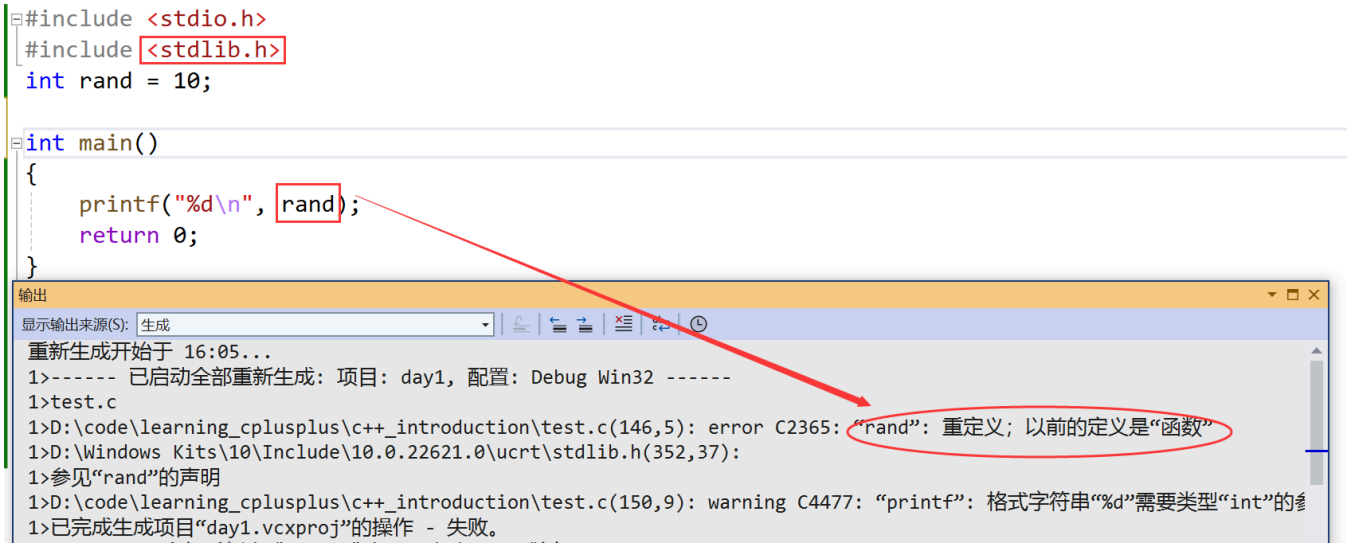
二、命名空间

命名空间介绍

- C++ 的命名空间（Namespace）是一种用于组织和管理代码标识符（变量、函数、类等）的方法。它可以避免命名冲突，使得在不同的命名空间中可以使用相同的标识符而不会产生命名冲突。
- 使用命名空间可以将一组相关的标识符封装在一起，提供了更好的代码组织和模块化能力。在 C++ 中，可以通过关键字 `namespace` 定义一个命名空间，并在其中声明相关的标识符。

下面定义了一个**全局变量rand**，而我还引入了头文件**stdlib**，而这个`rand`是`stdlib`中的一个库函数，-->rand，这样我想打印这个rand的值就会与库函数里的**rand**产生冲突，产生**重定义**，这个就是C语言中的不足之处了

```
#include <stdio.h>
#include <stdlib.h>
int rand = 10;

int main()
{
    printf("%d\n", rand);
    return 0;
}
```



- 在C++中就可以很好的解决这种问题：

> 定义命名空间，需要使用到**namespace**关键字，后面跟命名空间的名字，然后接一对{}即可，{}中即为命名空间的成员。

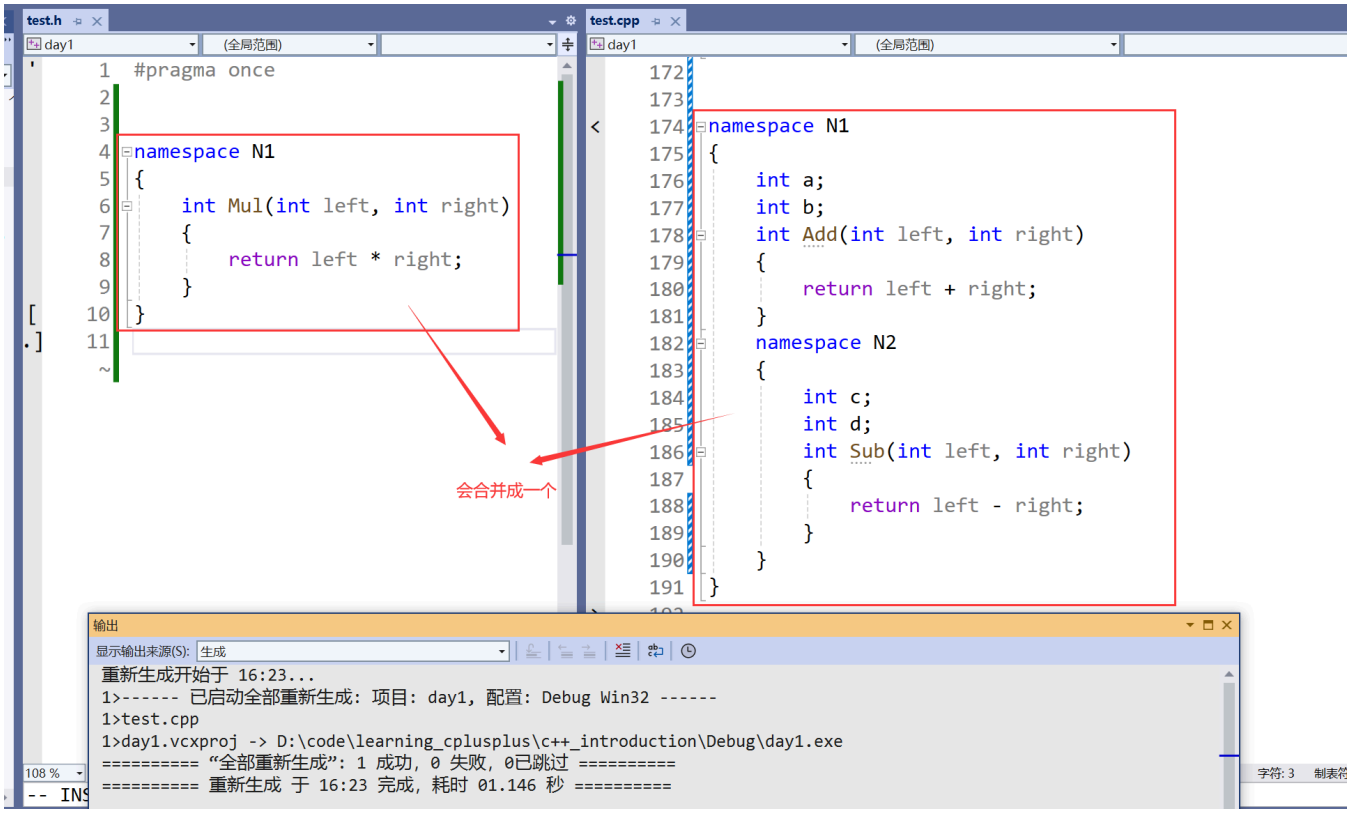
- 首先第一个正常的命名空间定义，也就是**namespace**后面跟上一个名字【随便起】，也就相当于是一个域，就好像C语言中的局部域，全局域。

```
namespace lsl
{
    // 命名空间中可以定义变量/函数/类型
    int rand = 10;

    int Add(int left, int right)
    {
        return left + right;
    }

    struct Node
    {
        struct Node* next;
        int val;
    };
}
```

- 第二个就是命名空间可以嵌套
- 这个也可以很好的理解，就和函数嵌套一样，一个域是一个域的，互不干扰

```
namespace N1
{
    int a;
    int b;
    int Add(int left, int right)
    {
        return left + right;
    }
    namespace N2
    {
        int c;
        int d;
        int Sub(int left, int right)
        {
            return left - right;
        }
    }
}
```

- 第三个就是同一个工程中允许存在多个相同名称的命名空间，编译器最后会合成同一个命名空间中。

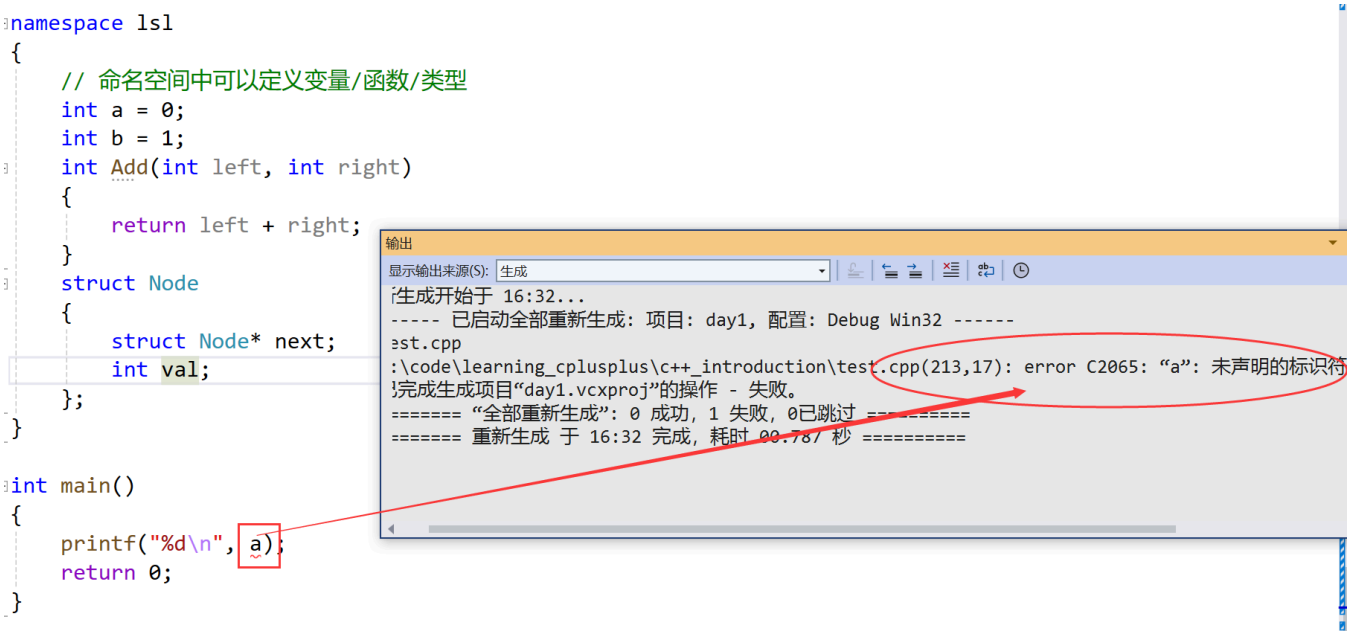


> 注意：一个命名空间就定义了一个新的作用域，命名空间中的所有内容都局限于该命名空间中

- 那么我这个**命名空间域**如何使用呢？
- 来看下面这段代码

```
namespace lsl
```

```
{
    // 命名空间中可以定义变量/函数/类型
    int a = 0;
    int b = 1;
    int Add(int left, int right)
    {
        return left + right;
    }
    struct Node
    {
        struct Node* next;
        int val;
    };
}
int main()
{
    printf("%d\n", a);
    return 0;
}
```



- 上面图片清晰的展示到，是不能这样用的，已经是在一个域里面了，所以就要使用特定的方法来使用这个域里面变量，函数和类型

命名空间的使用

> 命名空间的使用有三种方式：

- 第一种方式**加命名空间名称及作用域限定符【::】也就是两个冒号，也就是域作用限定符**

```c
int main()
{
    printf("%d\n", lsl::a);
    return 0;
}
```

```c
namespace lsl
{
    // 命名空间中可以定义变量/函数/类型
    int a = 0;
    int b = 1;
    int Add(int left, int right)
    {
        return left + right;
    }
    struct Node
    {
        struct Node* next;
        int val;
    };
}

int main()
{
    printf("%d\n", lsl::a);
    return 0;
}
```

- 第二种方式**使用using将命名空间中某个成员引入**

```
namespace lsl
{
    // 命名空间中可以定义变量/函数/类型
    int a = 0;
    int b = 1;
    int Add(int left, int right)
    {
        return left + right;
    }
    struct Node
    {
        struct Node* next;
        int val;
    };
}

using lsl::b;

int main()
{
    printf("%d\n", lsl::a);

    printf("%d\n", b);

    return 0;
```

指定域访问

指定成员

```
Microsoft Visual Studio 调试控制台
0
1

D:\code\learning_cplusplus\c++_introductio
按任意键关闭此窗口. . .
```

- 第三种方式就是使用**using namespace**命名空间名称引入，也就是展开，让**指定的命名空间里**的内容在外面都可以访问

```
namespace lsl
{
    // 命名空间中可以定义变量/函数/类型
    int a = 0;
    int b = 1;
    int Add(int left, int right)
    {
        return left + right;
    }
    struct Node
    {
        struct Node* next;
        int val;
    };
}

using namespace lsl;

int main()
{
    printf("%d\n", lsl::a);

    printf("%d\n", b);

    return 0;
}
```

```
Microsoft Visual Studio 调试控制台
0
1

D:\code\learning_cplusplus\c++_
按任意键关闭此窗口. . .
```

> 这里的最后一种方式相信大家学c++或多或少也都见到过，平常在使用c++的时候都要有这么一句**using namespace std;**，`std`是C++标准库的命名空间名，C++将标准库的定义实现都放到这个命名空间中，这里的展开和头文件的展开是不一样的，不要混淆了~~

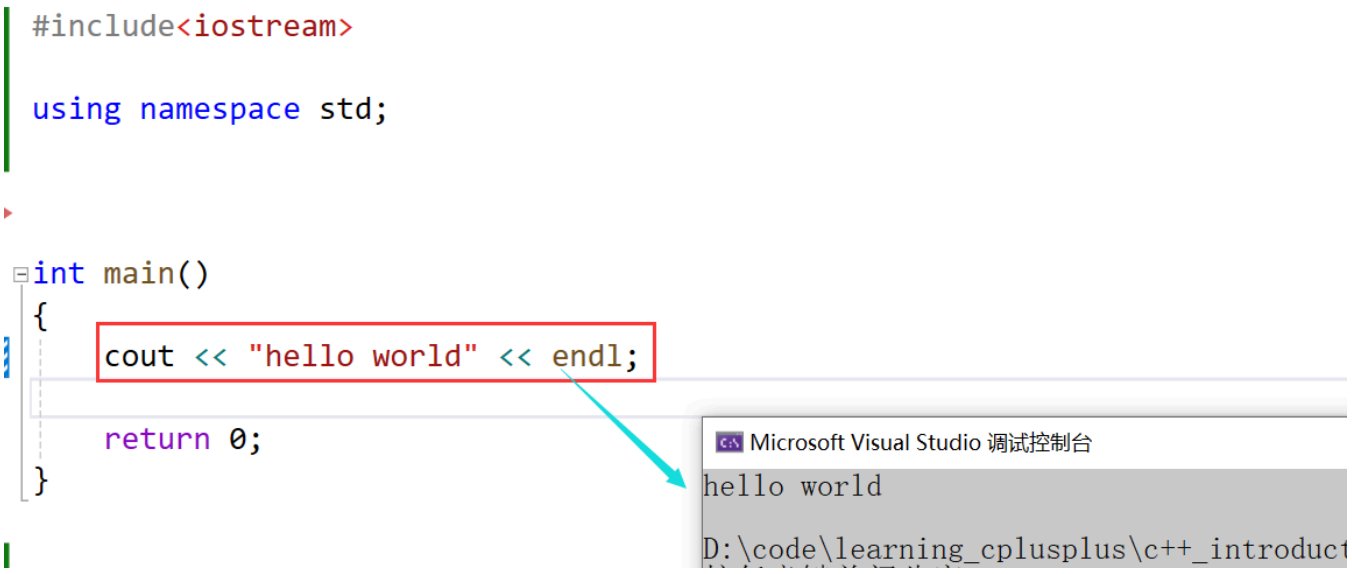
> 到这里我们也知道了，首先是先从局部域来搜索，然后从全局域再搜索，搜索不到就报错了，然而我们这里新增加了一个域作用限定符【::】，有了这个就是指定域搜索。

- 编译器搜索原则：

> **不指定域**：当前局部域，全局域
>
> 指定域：如果指定了，直接去指定域搜索

三、C++输入【cin】& 输出【cout】

```
#include<iostream>
using namespace std;
int main()
{
    cout << "hello world" << endl;
    return 0;
}
```

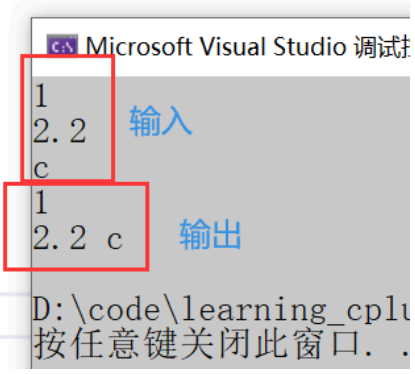


说明：

1. 使用cout标准输出对象(控制台)和cin标准输入对象(键盘)时，必须包含 < iostream >头文件以及按命名空间使用方法使用**std**
2. **cout**和**cin**是全局的流对象，**endl**是特殊的C++符号，表示**换行输出**，他们都包含在包含头文件中。
3. `<<`是**流插入运算符**，`>>`是**流提取运算符**。
4. 使用C++输入输出更方便，不需要像**printf/scanf**输入输出时那样，需要手动控制格式。**C++的输入输出可以自动识别变量类型。**

- 注意：**早期标准库将所有功能在全局域中实现，声明在.h后缀的头文件中，使用时只需包含对应头文件即可，后来将其实现在std命名空间下，为了和C头文件区分，也为了正确使用命名空间，规定C++头文件不带.h；旧编译器(vc 6.0)中还支持<iostream.h>格式，后续编译器已不支持，因此推荐使用**+std的方式。**
- 这里的**cout**【输出】，**cin**【输入】可以自动识别类型

```
#include <iostream>
using namespace std;
int main()
{
    int a;
    double b;
    char c;
    // 可以自动识别变量的类型
    cin >> a;
    cin >> b >> c;
    cout << a << endl;
    cout << b << " " << c << endl;
    return 0;
}
```



std命名空间的使用惯例：

> std是C++标准库的命名空间，如何展开std使用更合理呢？

1. 在日常练习中，建议直接**using namespace std**即可，这样就很方便。
2. **using namespace std**展开，标准库就全部暴露出来了，如果我们定义跟库重名的类型/对象/函数，就存在冲突问题。该问题在日常练习中很少出现，但是项目开发中代码较多、规模大，就很容易出现。所以建议在项目开发中使用，像**std::cout**这样使用时指定命名空间 + **using std::cout**展开常用的库对象/类型等方式。

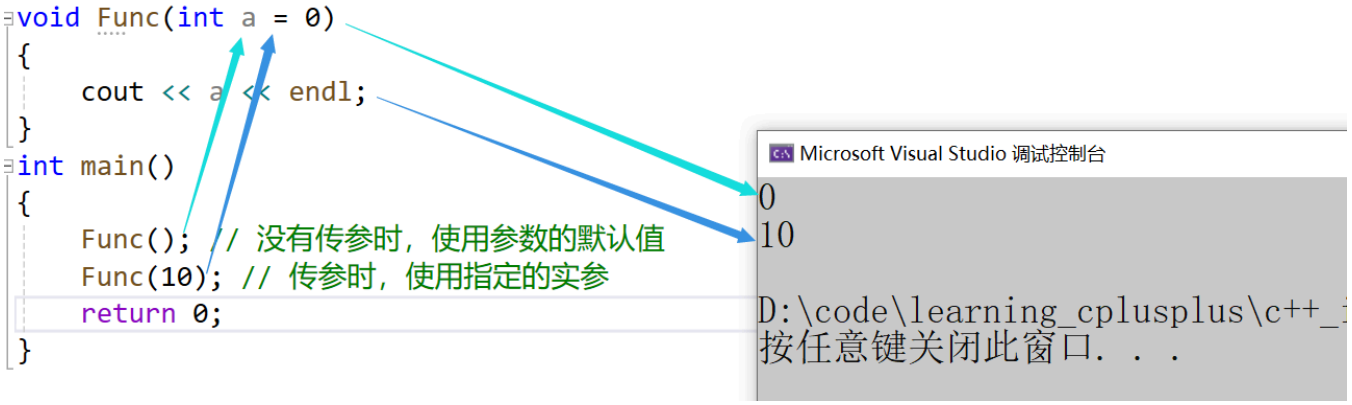
四、缺省参数

缺省参数概念

- 缺省参数是**声明或定义函数时为函数的参数指定一个缺省值。**
- 在调用该函数时，如果**没有指定实参则采用该形参的缺省值，否则使用指定的实参。**

接下来看一个案例就能明白了：

```cpp
void Func(int a = 0)
{
    cout << a << endl;
}
int main()
{
    Func(); // 没有传参时，使用参数的默认值
    Func(10); // 传参时，使用指定的实参
    return 0;
}
```



缺省参数分类

- 全缺省参数【也就是从左到右形参都有默认值】

```cpp
void Func(int a = 10, int b = 20, int c = 30)
{
    cout<<"a = "<<a<<endl;
    cout<<"b = "<<b<<endl;
    cout<<"c = "<<c<<endl;
}
```

- 半缺省参数【**必须是从右到左赋值**，不可以从左到右】

```cpp
void Func(int a, int b = 10, int c = 20)
{
    cout << "a = " << a << endl;
    cout << "b = " << b << endl;
    cout << "c = " << c << endl;
}
```

缺省参数的使用

- 我们可以这样使用：
- 第一个参数就是匹配的一个形参，依次匹配，**不可跳跃的传参**

```
void Func(int a = 10, int b = 20, int c = 30)
{
    cout << "a = "<< a <<endl;
    cout << "b = "<< b <<endl;
    cout << "c = " << c << endl << endl;
}

int main()
{
    Func(1,2,3);
    Func(2,3);
    Func(1);
    Func();
    return 0;
}
```

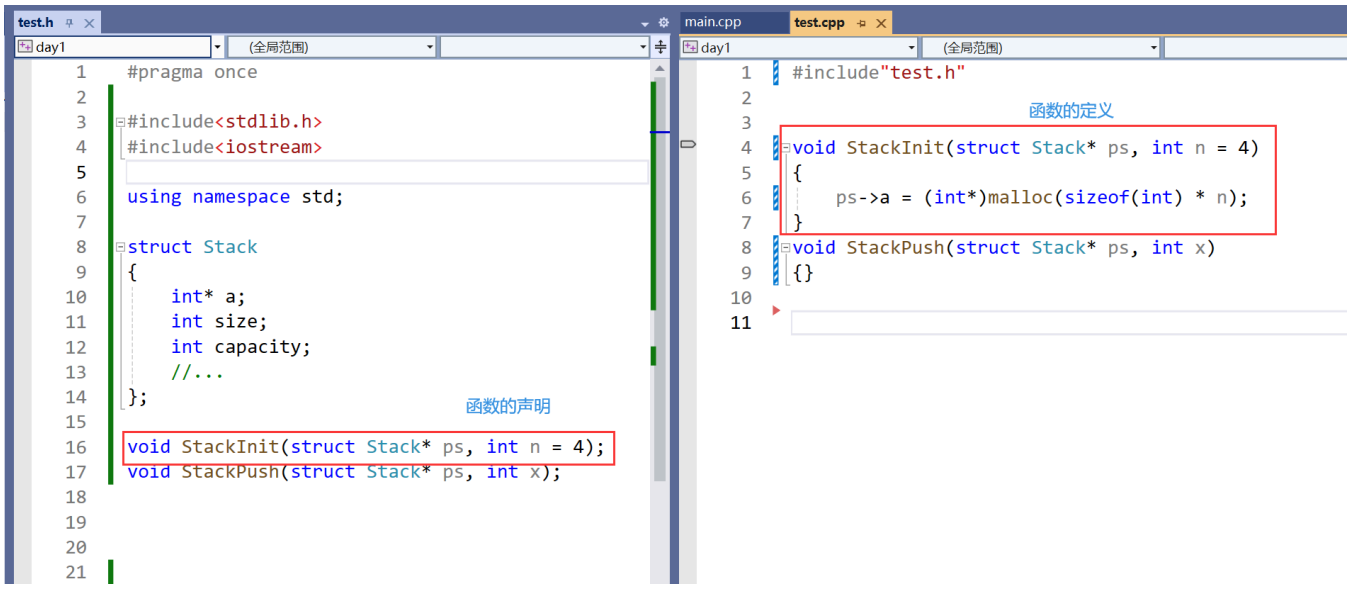
Microsoft Visual Studio 调试控制台

```
a = 1
b = 2
c = 3

a = 2
b = 3
c = 30

a = 1
b = 20
c = 30

a = 10
b = 20
c = 30
```

- 实际上我们是还可以这样用的，比如说我是定义了一个栈，栈在初始化的的时候不知道一开始初始化多少个，这个时候我们就可以使用缺省参数，指定传参。
- 比如我知道要插入100个数据，我就直接**指定100的大小就可以**，减少**扩容消耗**
- 如果不知道多少个就默认4个就可以啦~~

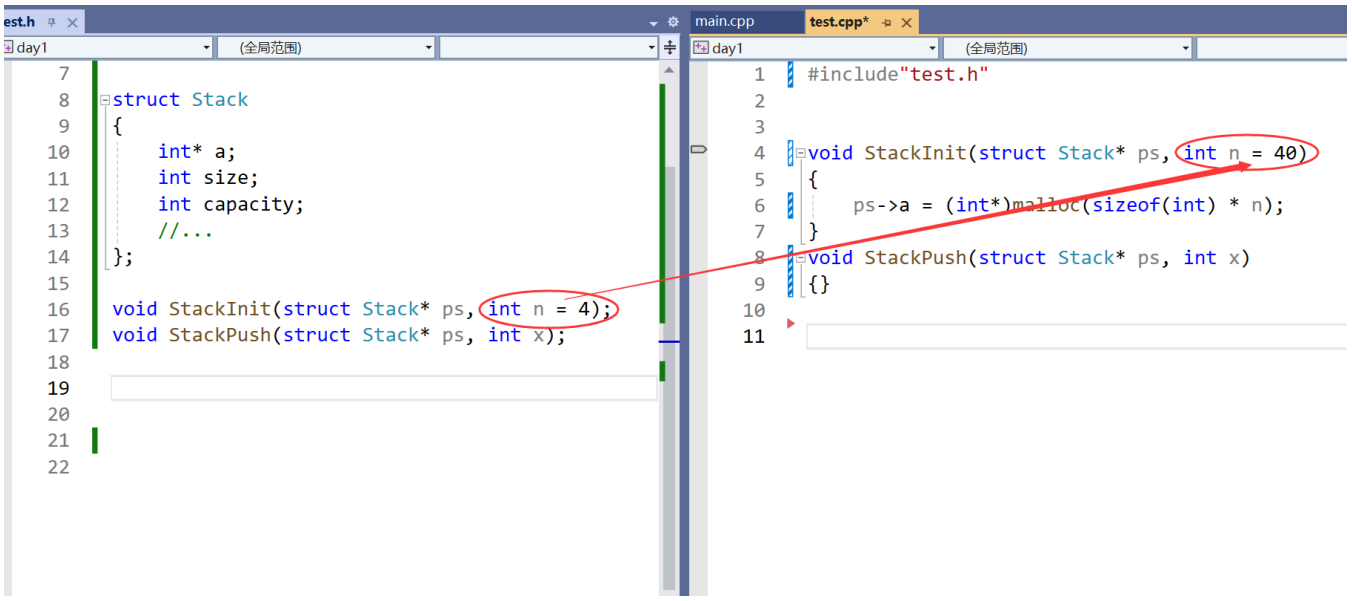
```c
#include "test.h"
struct Stack
{
    int* a;
    int size;
    int capacity;
    //...
};

void StackInit(struct Stack* ps, int n = 4)
{
    ps->a = (int*)malloc(sizeof(int) * n);
}
void StackPush(struct Stack* ps, int x)
{}


int main()
{
    struct Stack st1;

    // 1、确定要插入100个数据
    StackInit(&st1, 100);

    // 2、只插入10个数据
    struct Stack st2;
    StackInit(&st2, 10);

    // 3、不知道要插入多少个
    struct Stack st3;
    StackInit(&st3);

    return 0;
}
```

我们这里可以分文件写这个栈



- 我们这里的**声明**和**定义**可以同时写吗？

> 我们有人写代码的时候就有可能写成这样的操作，声明的参数和定义的参数不一样，那么是依谁的为准呢？



- 我们这里直接给出结论，是必须在**声明**给，为什么呢？
  - 这就需要了解代码的**编译原理**了，可以看一下C语言的**编译程序环境和预处理，程序编译+链接全过程**
  - 在同一文件下，缺省值**在函数的声明与定义中同时定义**，在编译阶段的语法检查时，直接报错了~

小结一下

1. **半缺省参数必须从右往左依次来给出，不能间隔着给**
2. **缺省参数不能在函数声明和定义中同时出现**
3. **缺省值必须是常量或者全局变量**
4. **C语言不支持（编译器不支持）**

五、函数重载

函数重载介绍

- **函数重载**：是函数的一种特殊情况，C++允许在同一作用域中声明几个功能类似的同名函数，这些同名函数的形参列表（`参数个数` 或 `类型` 或 `类型顺序`）不同，常用来处理实现功能类似数据类型不同的问题。

函数重载类型

> 函数重载有3个类型，我们分别来看一下：

- 第一个就是**参数类型不同**

```
// 1、参数类型不同
int Add(int left, int right)
{
    cout << "int Add(int left, int right)" << endl;
    return left + right;
}
double Add(double left, double right)
{
    cout << "double Add(double left, double right)" << endl;
    return left + right;
}

int main()
{
    Add(10, 20);
    Add(10.1, 20.2);
```

```
选择 Microsoft Visual Studio 调试控制台
int Add(int left, int right)
double Add(double left, double right)

D:\code\learning_cplusplus\c++_introductio
按任意键关闭此窗口. . .
```

- 第二个是**参数个数不同**

```
// 2、参数个数不同
void f()
{
    cout << "f()" << endl;
}

void f(int a)
{
    cout << "f(int a)" << endl;
}

int main()
{
    f();
    f(10);
```

```
Microsoft Visual Studio 调试控制台
f()
f(int a)

D:\code\learning_cp
按任意键关闭此窗口.
```

- 第三个就是**参数类型顺序不同**

```
// 3、参数类型顺序不同
void f(int a, char b)
{
    cout << "f(int a,char b)" << endl;
}

void f(char b, int a)
{
    cout << "f(char b, int a)" << endl;
}

int main()
{
    f(10, 'a');
    f('a', 10);
```

```
Microsoft Visual Studio 调试控制台
f(int a,char b)
f(char b, int a)

D:\code\learning_cplusplus\
按任意键关闭此窗口. . .
```

- 最后这里要注意：**构成函数重载的条件是** `函数名相同`，`参数不同`，**构成函数重载**【非常重要】

下面我们来看一道题【自己先做一下】来巩固一下：

> 以下不是double compare(int,int)的重载函数的是( )

> **A. int compare(double,double)**
> **B. double compare(double,double)
> C. double compare(double,int)
> D. int compare(int,int)**

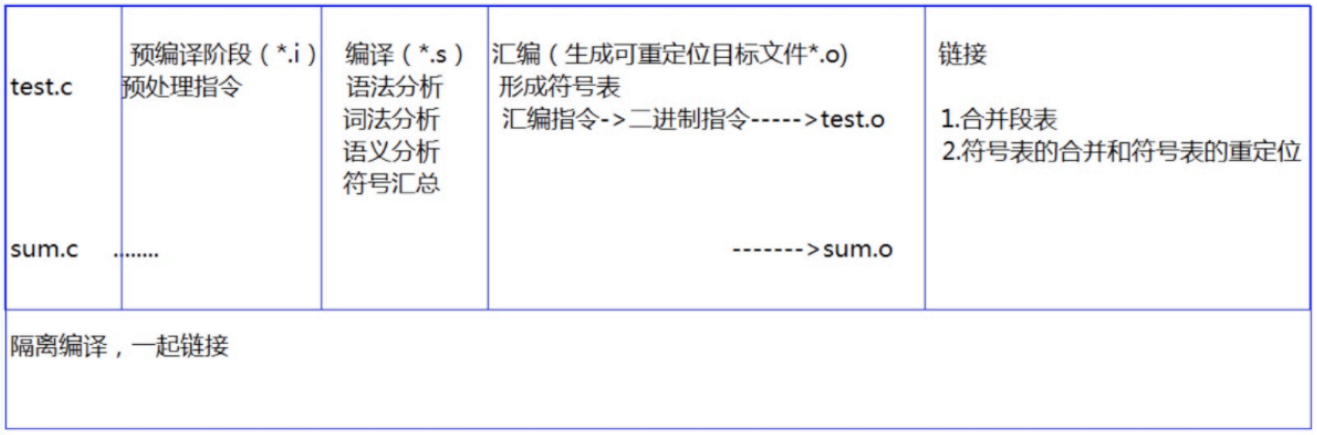
解析：

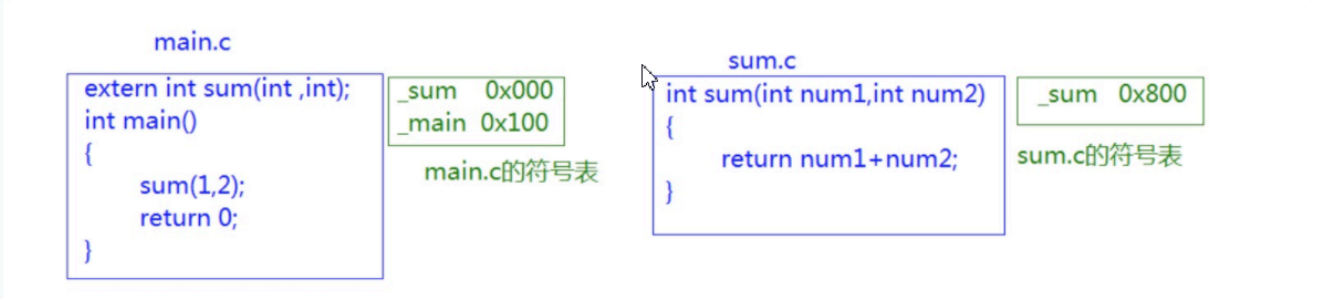
> A.重载必须是参数列表有所不同(包括个数和类型)，所以参数类型不同，构成重载
> B.参数类型不同，构成重载
> C.参数类型不同，构成重载
> D.函数重载不能依靠返回值的不同来构成重载，因为调用时无法根据参数列表确定调用哪个重载函数，故错误

六、C++支持函数重载的原理--名字修饰(name Mangling)【重点】

> 那么为什么C++支持**函数重载**呢？而C语言不支持呢？

- 靠的就是**名字修饰**





1. 实际项目通常是由多个头文件和多个源文件构成，而通过C语言阶段学习的编译链接，我们可以知道，【当前a.cpp中调用了b.cpp中定义的Add函数时】，编译后链接前，`a.o`的目标文件中没有Add的函数地址，因为Add是在`b.cpp`中定义的，所以Add的地址在`b.o`中。那么怎么办呢？
2. 所以链接阶段就是专门处理这种问题，链接器看到`a.o`调用Add，但是没有Add的地址，就会到`b.o`的符号表中找Add的地址，然后链接到一起。
3. 那么链接时，面对Add函数，链接接器会使用哪个名字去找呢？这里每个编译器都有自己的函数名修饰规则。
4. 由于Windows下vs的修饰规则过于复杂，而Linux下g++的修饰规则简单易懂，下面我们使用了g++演示了这个修饰后的名字。
5. 通过下面我们可以看出gcc的函数修饰后名字不变。而g++的函数修饰后变成【_Z+函数长度+函数名+类型首字母】。

```
[lin@shilin file111]$ cat test.c

int Add(int a,int b)
{
  return a+b;
}

void func(int a,double b,int* p)
{}

int main()
{
  Add(1,2);
  func(1,2,0);
  return 0;
}

[lin@shilin file111]$
```

gcc 编译后的结果:

```
00000000004004ed <Add>:
  4004ed:       55                      push   %rbp
  4004ee:       48 89 e5                mov    %rsp,%rbp
  4004f1:       89 7d fc                mov    %edi,-0x4(%rbp)
  4004f4:       89 75 f8                mov    %esi,-0x8(%rbp)
  4004f7:       8b 45 f8                mov    -0x8(%rbp),%eax
  4004fa:       8b 55 fc                mov    -0x4(%rbp),%edx
  4004fd:       01 d0                   add    %edx,%eax
  4004ff:       5d                      pop    %rbp
  400500:       c3                      retq

0000000000400501 <func>:
  400501:       55                      push   %rbp
  400502:       48 89 e5                mov    %rsp,%rbp
  400505:       89 7d fc                mov    %edi,-0x4(%rbp)
  400508:       f2 0f 11 45 f0          movsd  %xmm0,-0x10(%rbp)
  40050d:       48 89 75 e8             mov    %rsi,-0x18(%rbp)
  400511:       5d                      pop    %rbp
  400512:       c3                      retq

0000000000400513 <main>:
  400513:       55                      push   %rbp
  400514:       48 89 e5                mov    %rsp,%rbp
  400517:       48 83 ec 08             sub    $0x8,%rsp
  40051b:       be 02 00 00 00          mov    $0x2,%esi
  400520:       bf 01 00 00 00          mov    $0x1,%edi
  400525:       e8 c3 ff ff ff          callq  4004ed <Add>
  40052a:       48 b8 00 00 00 00 00    movabs $0x4000000000000000,%rax
```

结论：**在linux下，采用gcc编译完成后，函数名字的修饰没有发生改变，之所以就函数名不能相同**

采用**g++编译器**编译后结果：

```
000000000040050d <_Z3Addii>:
  40050d:       55                      push   %rbp
  40050e:       48 89 e5                mov    %rsp,%rbp
  400511:       89 7d fc                mov    %edi,-0x4(%rbp)
  400514:       89 75 f8                mov    %esi,-0x8(%rbp)
  400517:       8b 45 f8                mov    -0x8(%rbp),%eax
  40051a:       8b 55 fc                mov    -0x4(%rbp),%edx
  40051d:       01 d0                   add    %edx,%eax
  40051f:       5d                      pop    %rbp
  400520:       c3                      retq

0000000000400521 <_Z4funcidPi>:
  400521:       55                      push   %rbp
  400522:       48 89 e5                mov    %rsp,%rbp
  400525:       89 7d fc                mov    %edi,-0x4(%rbp)
  400528:       f2 0f 11 45 f0          movsd  %xmm0,-0x10(%rbp)
  40052d:       48 89 75 e8             mov    %rsi,-0x18(%rbp)
  400531:       5d                      pop    %rbp
  400532:       c3                      retq

0000000000400533 <main>:
  400533:       55                      push   %rbp
  400534:       48 89 e5                mov    %rsp,%rbp
  400537:       48 83 ec 08             sub    $0x8,%rsp
  40053b:       be 02 00 00 00          mov    $0x2,%esi
  400540:       bf 01 00 00 00          mov    $0x1,%edi
  400545:       e8 c3 ff ff ff          callq  40050d <_Z3Addii>
  40054a:       48 b8 00 00 00 00 00    movabs $0x4000000000000000,%rax
```

结论：**在`linux`下，采用`g++`编译完成后，函数名字的修饰发生改变，编译器将函数参数类型信息添加到修改后的名字中，所以就支持函数重载**

- Windows下**名字修饰规则**

表 3-19

函数签名	修饰后名称
int func(int)	?func@@YAHH@Z
float func(float)	?func@@YAMM@Z
int C::func(int)	?func@C@@AAEHH@Z
int C::C2::func(int)	?func@C2@C@@AAEHH@Z
int N::func(int)	?func@N@@YAHH@Z
int N::C::func(int)	?func@C@N@@AAEHH@Z

我们以 int N::C::func(int)这个函数签名来猜测 Visual C++的名称修饰规则（当然，你只须大概了解这个修饰规则就可以了）。修饰后名字由“?”开头，接着是函数名由“@”符号结尾的函数名；后面跟着由“@”结尾的类名“C”和名称空间“N”，再一个“@”表示函数的名称空间结束；第一个“A”表示函数调用类型为“__cdecl”（函数调用类型我们将在第 4 章详细介绍），接着是函数的参数类型及返回值，由“@”结束，最后由“Z”结尾。可以看到函数名、参数的类型和名称空间都被加入了修饰后名称，这样编译器和链接器就可以区别同名但不同参数类型或名字空间的函数，而不会导致 link 的时候函数多重定义。

- 对比Linux会发现，windows下vs编译器对函数名字修饰规则相对复杂难懂，但道理都是类似的，我们就不做细致的研究了。
- **【扩展学习：C/C++函数调用约定和名字修饰规则，里面有对vs下函数名修饰规则讲解】**
- 通过这里就理解了**C语言没办法支持重载**，因为**同名函数没办法区分。**而C++是通过**函数修饰规则来区分**，**只要参数不同**，**修饰出来的名字就不一样，就支持了重载。**
- 如果两个函数函数名和参数是一样的，返回值不同是不构成重载的，因为调用时编译器没办法区分。